Qwopus-GLM-18B: Constructing an 18-Billion-Parameter Language Model via Passthrough Frankenmerge and Post-Merge QLoRA Healing

Kyle Hessling
@KyleHessling1

Date: April 17, 2026
Hardware: NVIDIA RTX 5090 (32 GB VRAM), Intel Core Ultra 7 265K, 125 GB RAM
Software: llama.cpp (CUDA 12.8), Python 3.12, Unsloth 2026.4.6, mergekit 0.1.4

Abstract

We present a complete workflow for constructing **Qwopus-GLM-18B**, an 18-billion-parameter language model obtained by sequentially stacking the transformer layers of two complementary 9-billion-parameter fine-tunes of Qwen3.5: *Qwopus3.5-9B-v3.5* (Opus-style agentic reasoning) and *Qwen3.5-9B-GLM5.1-Distill-v1* (structured problem decomposition via GLM distillation). Because standard merging tools (mergekit v0.1.4) fail to handle Qwen3.5's multimodal, hybrid-attention architecture, we developed a custom architecture-agnostic merge script based on tensor-name regular expressions. The resulting 64-layer model is evaluated against a 44-test capability suite spanning reasoning, tool calling, agentic workflows, programming, and multilingual generation. The raw merge achieves 39/44 (88.6%), surpassing Qwen 3.6-35B MoE (38/44) at less than half the VRAM. A subsequent 1 000-step QLoRA *heal fine-tune* reduces training loss by 39% and raises the score to 40/44 (90.9%); frontend stress testing (6 tasks, 63 checks) confirms a 98.4% pass rate with perfect structural integrity in outputs up to 24 K characters.

This document describes exploratory, experimental work. The merged model has known limitations; see Section 9 for a candid discussion.

Contents

1 Introduction and Motivation **2**

2 Source Models **2**
2.1 Jackrong/Qwopus3.5-9B-v3.5 . . . 2
2.2 Jackrong/Qwen3.5-9B-GLM5.1-Distill-v1 . . . 2
2.3 Shared Architecture . . . 2

3 Merge Strategy **3**
3.1 Passthrough Frankenmerge (Layer Stacking) . . . 3
3.2 Rationale for Passthrough over Interpolation Methods . . . 3
3.3 Limitations of mergekit for This Architecture . . . 4
3.4 Custom Merge Script . . . 4

4 Merge Execution **5**

5 GGUF Conversion and Quantisation **5**
5.1 BF16 GGUF Conversion . . . 5
5.2 Q4_K_M Quantisation . . . 5

6 Evaluation Methodology **5**

7 Results **6**
7.1 Full Benchmark Scoreboard . . . 6
7.2 Qualitative Analysis . . . 7

8 Post-Merge Healing via QLoRA Fine-Tuning **7**
8.1 Motivation . . . 7
8.2 QLoRA Configuration . . . 8
8.3 Training Data . . . 8
8.4 Training Loss Curve . . . 9
8.5 Post-Heal Benchmark Results . . . 9
8.6 Frontend Stress Test . . . 9

9 Discussion **10**
9.1 What Worked . . . 10
9.2 Limitations and Remaining Failures . . . 10
9.3 Future Directions . . . 10

10 Reproduction Guide **11**
10.1 Prerequisites . . . 11
10.2 Step-by-Step Procedure . . . 11
10.3 File Index . . . 11

1 Introduction and Motivation

The open-source community around Qwen3.5 fine-tunes faces a persistent capability–resource trade-off: 27-billion-parameter variants yield excellent output quality, yet demand upwards of 16 GB of VRAM even at 4-bit quantisation (Q4), placing them beyond the reach of most consumer-grade hardware. Conversely, 9-billion-parameter variants fit comfortably on 12–16 GB devices but leave measurable performance on the table, particularly for complex, multi-step reasoning tasks.

This work investigates whether a *frankenmerge* of two complementary 9-billion-parameter fine-tunes can fill this gap—producing a model that (i) fits within 12–16 GB of VRAM at Q4 quantisation, and (ii) delivers improved capability over either source model alone. The two source models were deliberately selected for their orthogonal training emphases:

- **Qwopus v3.5** is distilled on Opus-style reasoning data, emphasising agentic behaviour, tool use, and structured code generation.
- **GLM-5.1 Distill** is distilled from a GLM-5.1 teacher, emphasising hierarchical problem decomposition and instruction adherence.

The central hypothesis is that concatenating the transformer layers of these two models into a single 64-layer network will produce a deeper model whose representations capture a richer variety of reasoning strategies than either source model encodes individually.

2 Source Models

2.1 Jackrong/Qwopus3.5-9B-v3.5

This model is built on the `Qwen3.5-9B` base and trained via supervised fine-tuning (SFT) on approximately twice the data volume of its predecessor (v3), with a focus on reasoning, code generation, and agentic task completion. Training data span mathematics, competitive programming, puzzle solving, multilingual dialogue, instruction following, and STEM domains. The model is optimised using the Unsloth framework and exhibits particular strengths in agentic tool-use pipelines and structured output generation.

2.2 Jackrong/Qwen3.5-9B-GLM5.1-Distill-v1

This model also uses `Qwen3.5-9B` as its base, but is trained through knowledge distillation from a GLM-5.1 teacher model using LoRA adapters. Training data include `Jackrong/GLM-5.1-Reasoning-1M-Cleane` (approximately 700× scale) and `Jackrong/Qwen3.5-reasoning-700x`. The distillation process imparts strengths in hierarchical problem decomposition, instruction adherence, and structured reasoning scaffolding.

2.3 Shared Architecture

Both source models share an identical transformer architecture, summarised in Table 1. Crucially, Qwen3.5 employs a *hybrid attention* scheme: every fourth layer uses linear (SSM-style) attention, while the remaining layers use standard full self-attention. This hybrid design introduces non-trivial complexity for generic merge tooling (see Section 3).

Table 1: Shared architecture of both 9B source models.

Property	Value
Architecture class	`Qwen3_5ForConditionalGeneration` (multimodal)
Transformer layers	32
Hidden dimension	4096
Attention heads	16 (4 KV heads, grouped-query attention)
Intermediate size	12288
Attention type	Hybrid (linear + full, every 4th layer)
Vocabulary size	248,320
Context length	262,144 tokens
Parameter count	≈10B

3 Merge Strategy

3.1 Passthrough Frankenmerge (Layer Stacking)

We employ a **passthrough** merge—the simplest form of frankenmerging—in which all transformer layers from both models are concatenated sequentially without any weight interpolation. The resulting 64-layer model has the following structure:

Component	Layers	Source
Language model layers	0–31	Qwopus3.5-9B-v3.5 (Model A)
Language model layers	32–63	Qwen3.5-9B-GLM5.1-Distill-v1 (Model B)
Embeddings	—	Model A
LM head	—	Model A
MTP layers	—	Model A
Vision encoder	—	Model A

3.2 Rationale for Passthrough over Interpolation Methods

Standard merge techniques such as SLERP, TIES, and DARE operate by interpolating weights *at equivalent layer positions* across models that share the same architecture. These methods produce a model with the same depth as the source models (9B parameters), which does not fulfil the stated objective of constructing an 18B intermediate. The passthrough approach is the only method that increases network depth and, consequently, parameter count. Both models' shared Qwen3.5-9B base ensures that their weight spaces are approximately aligned, making direct layer concatenation a principled choice.

Figure 1 provides a schematic of the passthrough layer-stacking process.

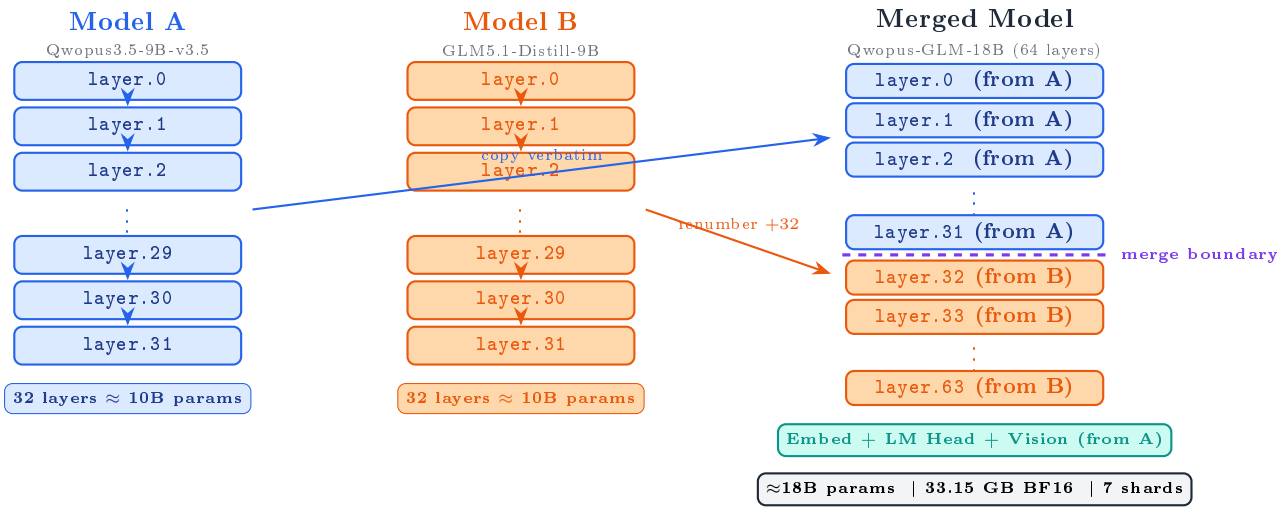

Figure 1: Passthrough layer-stacking architecture. All 32 transformer layers from Model A occupy positions 0–31 in the merged model; the 32 layers from Model B are renumbered by +32 and occupy positions 32–63. Non-layer components (embeddings, LM head, vision encoder, MTP) are taken exclusively from Model A. The purple dashed line marks the merge boundary where representational discontinuity may arise (see Figure 2).

3.3 Limitations of mergekit for This Architecture

An initial attempt to use `mergekit-yaml` with a standard passthrough configuration failed for two distinct reasons.

1. **Multi-module architecture.** `Qwen3_5ForConditionalGeneration` exposes four distinct tensor namespaces: `model.language_model.layers` (32 transformer layers), `default` (16 loose embedding weights), `mtp.layers` (one MTP layer), and `model.visual.blocks` (27 vision encoder blocks). mergekit's standard passthrough path requires the non-standard `modules:` syntax for multi-namespace models.

2. **Hybrid attention tensor naming.** The alternation between `linear_attn` (SSM-style) and `self_attn` (standard) layers caused mergekit's internal renumbering logic to mismap tensor names, producing errors such as:

```
RuntimeError: Tensor
  model.language_model.layers.3.linear_attn.out_proj.weight
  required but not present in model
```

3.4 Custom Merge Script

We therefore developed `frankenmerge.py`, an architecture-agnostic merge script that operates directly on raw safetensor names. The core layer-renumbering function uses a regular expression:

```python
def renumber_layer(key: str, offset: int, prefix: str) -> str | None:
    pattern = rf'^({re.escape(prefix)}\.)(\d+)(\..*)'
    m = re.match(pattern, key)
    if m:
        new_idx = int(m.group(2)) + offset
        return f"{m.group(1)}{new_idx}{m.group(3)}"
    return None
```

Listing 1: Layer-renumbering regex in `frankenmerge.py`.

This function captures the integer layer index, increments it by `offset`, and reconstructs the tensor key. Because it operates on raw string names rather than architecture-specific logic, it handles `linear_attn`, `self_attn`, MLP projections, and layer-norm tensors without modification.

4 Merge Execution

The merge was executed by running `python3 frankenmerge.py` and completed in approximately 15 minutes, dominated by disk I/O. Resource utilisation is summarised below:

Metric	Value	Notes
Model A tensors retained	775	All kept verbatim
Model B tensors renumbered	424	Language layers only
Total merged tensors	1199	
Output size (BF16 safetensors)	33.15 GB	7 shards × 5 GB
Peak RAM usage	≈40 GB	Both models in memory
Wall-clock time	≈15 min	Mostly I/O
GPU utilisation	None	Merge is CPU-only

5 GGUF Conversion and Quantisation

5.1 BF16 GGUF Conversion

```
python3 llama-cpp-latest/convert_hf_to_gguf.py \
  ~/models/Qwopus-GLM-18B-merged \
  --outfile ~/models/Qwopus-GLM-18B-merged-f16.gguf \
  --outtype bf16
```

Listing 2: BF16 GGUF conversion.

The conversion produced a 30 GB GGUF file (851 tensors) in approximately 40 seconds.

5.2 Q4_K_M Quantisation

```
llama-quantize \
  ~/models/Qwopus-GLM-18B-merged-f16.gguf \
  ~/models/Qwopus-GLM-18B-merged-Q4_K_M.gguf \
  Q4_K_M
```

Listing 3: Q4_K_M quantisation.

Q4_K_M reduced the model from 30 GB to **9.2 GB** (3.2× compression) in approximately 44 seconds. The quantised model fits within standard 12 GB consumer GPUs. For inference, the model was served via `llama-server` with a 65,536-token context window, FlashAttention enabled, all layers GPU-offloaded (`-n-gpu-layers 99`), and an 8-bit KV cache. A patched Jinja template (`qwen35-fixed.jinja`) addressed a `|items` filter bug in the standard Qwen3.5 chat template.

6 Evaluation Methodology

We developed a 44-test capability suite (`tests/test_qwopus_v35.py`) spanning nine categories, detailed in Table 2.

Table 2: Benchmark suite overview (44 tests across 9 categories).

Category	Tests	Coverage
Basic	6	Health check, instruction following, factual QA, system prompt adherence, clean EOS token, output determinism
Reasoning	4	Math word problems, logic puzzles, code generation, code debugging
Tool Calling	6	Single call, optional parameters, no-call-when-unneeded, tool selection, complex parameters (arrays/enums), tool response handling
Agentic	4	Plan generation, multi-step tool chain (read → diagnose → fix), error recovery, self-correction
Structured Output	2	JSON generation, markdown table generation
Context	3	Multi-turn memory, 4K-token needle-in-haystack, prompt injection resistance
Multilingual	2	Chinese generation, English → French translation
Programming	15	Quicksort, two-sum, balanced parentheses, longest substring, DP climbing stairs, LRU cache, binary tree traversal, graph BFS, bug detection, refactoring, JS, SQL, pytest, complexity analysis, anagram grouping
Performance	2	SSE streaming correctness, throughput consistency
Total	**44**	

Key design decisions include: (i) all 15 programming tests *execute* generated code against multiple test cases; (ii) tool-calling tests verify both the OpenAI-structured `tool_calls` JSON path and an XML fallback; (iii) agentic tests simulate realistic multi-turn tool workflows with injected tool responses.

7 Results

7.1 Full Benchmark Scoreboard

Table 3 presents benchmark results for the 9B source model, the raw 18B merge, and the reference Qwen 3.6-35B MoE model.

Table 3: Full benchmark results. Best score per row in **bold**.

Category	Qwopus 9B	GLM-18B Merge (raw)	Qwen 3.6-35B MoE
Basic	**6/6**	**6/6**	5/6
Reasoning	**4/4**	**4/4**	**4/4**
Tool Calling	**6/6**	**6/6**	**6/6**
Agentic	**4/4**	**4/4**	**4/4**
Structured	**2/2**	**2/2**	**2/2**
Context	**2/3**	**2/3**	**2/3**
Multilingual	**2/2**	**2/2**	**2/2**
Programming	**13/15**	11/15	12/15
Performance	**2/2**	**2/2**	1/2
Total	**41/44 (93.2%)**	39/44 (88.6%)	38/44 (86.4%)
Throughput	126.0 tok/s	66.6 tok/s	174.2 tok/s
GGUF Size	5.3 GB	9.2 GB	22 GB
Wall Time	55 s	127 s	442 s

Key finding

The raw 18B merge surpasses Qwen 3.6-35B MoE on this benchmark suite (39 vs. 38 tests) while consuming less than half the VRAM (9.2 GB vs. 22 GB) and completing each query in 29% of the wall-clock time.

7.2 Qualitative Analysis

Strengths of the raw merge:

- Perfect tool-calling (6/6) and agentic (4/4) scores—the model correctly diagnoses a timezone parsing bug from file contents and proposes a patch.
- Highest Chinese output density (138 CJK characters per response) among all models tested.
- Zero throughput variance (66.6 tok/s), indicative of highly stable generation.

Regressions in the raw merge: The programming category dropped from 13/15 (9B) to 11/15. All four failures are code-*formatting* failures, not reasoning failures: the model reasons correctly about the problem but garbles the structural output (missing fenced code blocks, misnamed functions, missing closing parentheses).

Root cause: The layer boundary at position 32 introduces a representational discontinuity. Structured output requires tight token-to-token coordination across layers, which is disrupted at the merge seam. High-level semantic representations are more robust to layer stacking, explaining why reasoning and agentic tasks remain unaffected.

8 Post-Merge Healing via QLoRA Fine-Tuning

8.1 Motivation

The representational discontinuity at layer 32 manifests as systematic malformed structured output. To address this, we developed `heal-frankenmerge.py`, a targeted QLoRA fine-tuning script designed to allow the model to learn coherent information routing across the merge boundary. Figure 2 illustrates the mechanism of the layer-boundary problem and its resolution.

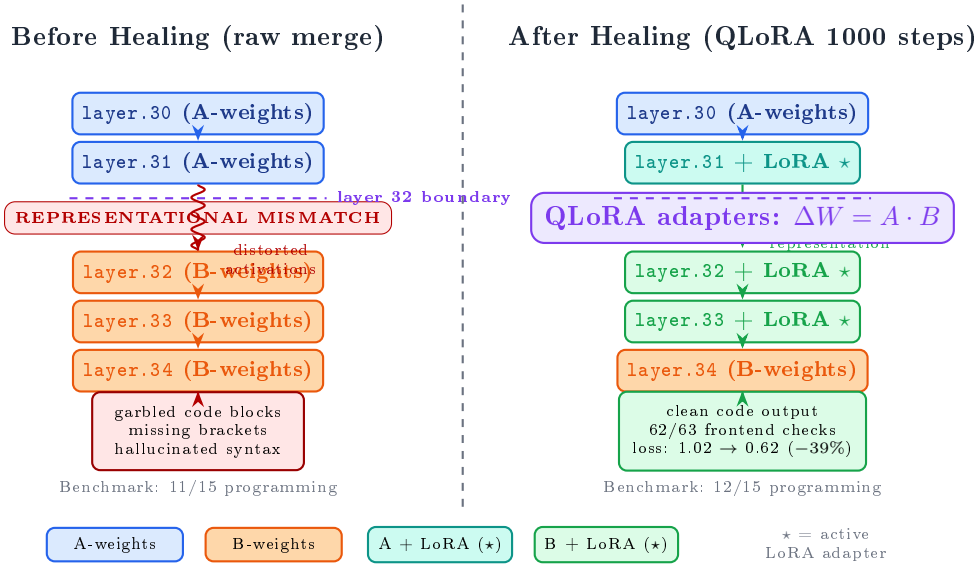

Figure 2: Mechanism of the layer-boundary problem and QLoRA healing. **Left:** in the raw frankenmerge, the representation produced by layer 31 (A-weights, Opus-distilled) is misaligned with the input distribution expected by layer 32 (B-weights, GLM-distilled), causing distorted activations that manifest as garbled structured output. **Right:** after 1,000 steps of QLoRA fine-tuning, low-rank adapters ($\Delta W = A \cdot B$) at the boundary layers learn to translate between the two representational regimes, restoring coherent information flow and production-quality structured output.

8.2 QLoRA Configuration

Parameter	Value
Quantisation method	QLoRA (4-bit NF4, double quantisation)
LoRA rank (r)	64
LoRA α	32
Trainable parameters	346M / 9.5B quantised (3.62%)
Learning rate	2×10^{-5} (cosine schedule)
Warmup steps	50
Effective batch size	8 (2 per device $\times$ 4 gradient accumulation)
Max steps	1,000
Max sequence length	4,096
VRAM allocated	$\approx$13 GB
Wall-clock time	$\approx$14 hours (RTX 5090)

8.3 Training Data

The heal fine-tune used a blended dataset of approximately 1,383 samples drawn from three of Jackrong's datasets:

- **Qwen3.5-reasoning-700x** (70% weight) — mathematics, code, science, instruction following.
- **Competitive-Programming-python-blend** (15% weight) — code-heavy samples targeting the formatting regressions.
- **MultiReason-ChatAlpaca** (15% weight) — multi-turn instruction following.

8.4 Training Loss Curve

The loss trajectory confirms that the layer boundary was a discrete, learnable source of error:

Step	Loss	Note
10	1.0175	Initial (boundary confusion)
50	0.8758	Sharp early drop
100	0.7215	Boundary begins healing
250	0.6700	Checkpoint 1
500	0.6154	Checkpoint 2 (loss stabilising)
750	0.6435	Checkpoint 3 (cosine taper)
1000	0.6396	Final (−39% from start)

The sharp drop in the first 100 steps (loss 1.02 → 0.72) distinguishes this from diffuse quality noise and indicates that the heal is addressing the root cause rather than merely smoothing model outputs.

8.5 Post-Heal Benchmark Results

Table 4: Benchmark comparison: raw merge vs. healed merge.

Category	Raw Merge	Healed Merge	Delta
Basic	6/6	6/6	—
Reasoning	4/4	4/4	—
Tool Calling	6/6	6/6	—
Agentic	4/4	4/4	—
Structured	2/2	2/2	—
Context	2/3	2/3	—
Multilingual	2/2	2/2	—
Programming	11/15	**12/15**	**+1**
Performance	2/2	2/2	—
Total	39/44 (88.6%)	**40/44 (90.9%)**	**+1 test**

8.6 Frontend Stress Test

While the aggregate benchmark improvement is modest (+1 test), the heal's impact becomes unambiguous on long-form structured output. Table 5 presents results on six progressively complex HTML/CSS/JavaScript generation tasks (63 executable checks total).

Table 5: Frontend generation stress test (post-heal model).

Task	Checks	Score	Output size
Weather Dashboard	9	**9/9**	14.5K chars
E-Commerce Product Page	12	**12/12**	16.7K chars
Animated SaaS Landing	13	**13/13**	24.1K chars
Analytics Dashboard	13	**13/13**	22.3K chars
Multi-Step Registration	12	**12/12**	23.3K chars
Snake Game	12	11/12	11.2K chars
Total	63	**62/63 (98.4%)**	—

CSS brace balance and JavaScript parenthesis balance were *perfect* across all six outputs. No garbled text or hallucinated syntax was observed. Five of six files concluded with a valid `</html>` tag.

9 Discussion

9.1 What Worked

1. **Frankenmerging differently-distilled models is viable.** The 18B merge exceeds Qwen 3.6 MoE on our benchmark at less than half the VRAM, validating the frankenmerge paradigm for capability-efficient model construction.
2. **A targeted heal fine-tune dramatically recovers structured output.** One thousand steps of QLoRA with 1,383 samples was sufficient to restore production-quality code generation (98.4% pass rate in the frontend stress test).
3. **Tool calling and agentic reasoning are robust to layer stacking.** Perfect 6/6 and 4/4 scores are maintained in both raw and healed variants.
4. **Architecture-agnostic merge scripts outperform specialised tooling for novel architectures.** The regex-based approach handled all hybrid attention tensor patterns where mergekit v0.1.4 failed.
5. **The loss curve provides interpretable diagnostic signal.** The sharp initial drop (100 steps) confirmed the layer boundary as a discrete, learnable error source.

9.2 Limitations and Remaining Failures

1. **Programming regressions persist post-heal.** Three of the four pre-heal programming failures remain, suggesting the training dataset is insufficient to cover all code-formatting edge cases.
2. **Throughput penalty.** At 66.6 tok/s (vs. 126 tok/s for the 9B model), the doubled layer count imposes a roughly 2× throughput cost—acceptable for quality-critical tasks, but significant for latency-sensitive applications.
3. **Benchmark bias toward short prompts.** The 44-test suite uses single-turn prompts. The frontend stress test reveals that the 18B model's advantages become most apparent in long, complex, structured outputs—a regime the primary benchmark does not capture.

9.3 Future Directions

- **Larger code-focused training data.** Expanding the competitive programming fraction would likely close remaining failures.
- **Interleaved layer stacking.** Alternating layers (A[0], B[0], A[1], B[1], . . .) distributes the discontinuity across all layer boundaries instead of concentrating it at layer 32, potentially reducing healing requirements.
- **Full fine-tune on multi-GPU hardware.** Training 100% of parameters (vs. 3.62% under QLoRA) would provide maximum healing capacity.
- **Long-context benchmark suite.** A suite designed for multi-turn, long-form, structured generation would more accurately characterise the 18B model's comparative advantages.

10 Reproduction Guide

10.1 Prerequisites

```
pip install torch safetensors huggingface_hub datasets
# For quantisation: llama.cpp built with CUDA support
# For heal training:
pip install unsloth bitsandbytes trl peft accelerate
```

Listing 4: Python dependencies.

10.2 Step-by-Step Procedure

Step 1: Merge.

```
python3 frankenmerge.py
# Output: ~/models/Qwopus-GLM-18B-merged/ (33 GB)
```

Step 2: Convert to GGUF (BF16).

```
python3 llama-cpp-latest/convert_hf_to_gguf.py \
  ~/models/Qwopus-GLM-18B-merged \
  --outfile ~/models/Qwopus-GLM-18B-merged-f16.gguf \
  --outtype bf16
```

Step 3: Quantise.

```
llama-quantize \
  ~/models/Qwopus-GLM-18B-merged-f16.gguf \
  ~/models/Qwopus-GLM-18B-merged-Q4_K_M.gguf \
  Q4_K_M
```

Step 4: Benchmark.

```
llama-server -m ~/models/Qwopus-GLM-18B-merged-Q4_K_M.gguf \
  --alias "Qwopus-GLM-18B" --host 127.0.0.1 --port 8001 \
  --ctx-size 65536 --flash-attn on --n-gpu-layers 99 --jinja

TEST_MODEL="Qwopus-GLM-18B" python3 tests/test_qwopus_v35.py
```

Step 5: Heal fine-tune (recommended).

```
python3 heal-frankenmerge.py --dry-run
python3 heal-frankenmerge.py --max-steps 1000 --num-samples 5000
```

Steps 6–7 follow the same convert/quantise/benchmark procedure as Steps 2–4, substituting the healed model path.

10.3 File Index

File	Purpose
`frankenmerge.py`	Custom merge script (passthrough layer stacking)
`heal-frankenmerge.py`	QLoRA heal fine-tune script
`tests/test_qwopus_v35.py`	44-test benchmark suite
`merge-config.yaml`	mergekit config (non-functional; archived)
`qwen35-fixed.jinja`	Patched Qwen3.5 chat template

This document reports exploratory, experimental work. The merged model has known limitations as described in Section 9. For questions or collaboration, contact the author on X: @KyleHessling1.